

Laisser un agent écrire ses propres outils

Exécuter du code non fiable dans une microVM, et ce que ça change pour un agent.

Denis Lamard

Concepteur et intégrateur d'agents IA

github.com/denislamard

Version 1.0, 2026

Sommaire

1. Le plafond des outils figés
2. Pourquoi « lancer le code » ne se fait pas
3. La microVM comme frontière
4. Un protocole minimal entre l'hôte et l'invité
5. Lier la session à la connexion
6. Le problème des dépendances
7. Limites de ressources, et le mot « plafond »
8. Ce que l'agent doit voir quand ça échoue
9. Les artefacts, et une décision qu'on ne reprend pas
10. Ce que ça change pour l'agent
11. Ce que cette hypothèse ne couvre pas
12. Ce que je retiens
13. À propos

1. Le plafond des outils figés

« Il pourrait pas le faire lui-même, l'agent ? »

La phrase vient d'un carreleur, au téléphone, après une démonstration. Il venait de voir un agent trier ses demandes de devis et relancer les clients silencieux, et il voulait maintenant que le même agent lui sorte le récapitulatif de ses chantiers du mois sous la forme exacte que lui réclame son comptable. Un tableau, deux colonnes de plus que ce que produit son logiciel, des totaux par client. Rien de compliqué. Rien de prévu non plus.

Un agent en production dispose d'un catalogue d'outils fermé. J'ai décrit dans le cahier 03 ce que ce catalogue contient et pourquoi il doit rester une liste écrite par une personne identifiable : des fonctions locales, des serveurs MCP, des appels à d'autres modèles. Ce choix est le bon. Il rend le périmètre lisible, il se relit avec le client, il se restreint. Il a aussi un plafond, et le carreleur venait de le toucher du doigt.

Chaque besoin nouveau passe par un développeur. Il faut écrire l'outil, le décrire, le tester, le déployer, et le client attend. Pour un cabinet de conseil qui facture l'intégration, c'est un revenu. Pour un artisan qui paie un abonnement de 40 euros par mois, c'est un délai que personne ne financera. Le récapitulatif pour le comptable ne sera jamais un outil du catalogue, parce qu'il ne concerne qu'un seul client et qu'il change à chaque comptable.

Ce que le modèle sait faire, pourtant, c'est écrire ce traitement. Une vingtaine de lignes de Python, un fichier en entrée, un fichier en sortie. Il l'écrit en quelques secondes, et il l'écrit à peu près juste. La question de ce cahier tient là : que faudrait-il pour laisser un agent écrire un outil qui n'existe pas encore, l'exécuter, et le réutiliser, sans mettre en danger la machine qui l'héberge ni les données des autres clients ?

Ce cahier est une réflexion. Il décrit un design, avec ses arbitrages, et il laisse ouvertes les questions que je n'ai pas tranchées. Les chiffres qu'il contient sont des ordres de grandeur retenus pour raisonner ; aucun ne vient d'un relevé de production. La partie 11 rassemble ce que cette hypothèse ne couvre pas.

Où placer la limite

Je ne cherche pas à remplacer le catalogue. Un outil qui envoie un courriel, qui écrit dans un CRM ou qui déclenche un paiement reste un outil écrit par un humain, décrit par un humain, et relu avec le client. Ce qui me manque, c'est une case pour le reste : les transformations locales, sans effet de bord, qui prennent des données en entrée et rendent un résultat ou un fichier. Le récapitulatif du carreleur en fait partie. Un export au format d'un logiciel de paie aussi. Le croisement de deux fichiers Excel pour trouver les clients facturés deux fois, encore.

La frontière que je propose est celle de l'effet de bord. Tout ce qui lit des entrées et produit des sorties, sans toucher au monde extérieur, peut être écrit par le modèle. Tout ce qui touche au monde reste dans le catalogue. Cette ligne est facile à énoncer. Elle est plus difficile à faire respecter, et c'est l'objet des parties qui suivent.

2. Pourquoi « lancer le code » ne se fait pas

La première tentation est de prendre le code produit par le modèle et de l'exécuter dans un sous-processus, sur le serveur qui héberge l'agent. Ça marche en démonstration. Ça marche encore la première semaine. Et puis un jour le modèle écrit `open("/etc/passwd")` par confusion, ou un `rmtree` sur le mauvais répertoire, ou une boucle qui remplit le disque. Sans malveillance. Par erreur, comme un développeur junior à qui on aurait donné les droits de production le premier jour.

Le code écrit par un modèle est du code non fiable au sens strict. Il n'a pas été relu, il a été produit à partir d'une demande formulée par un client, et cette demande peut elle-même contenir des instructions glissées dans un fichier joint. Le cahier 04 posait cette règle pour un chemin de fichier : une chaîne fournie par un modèle est une entrée hostile. Un programme entier fourni par un modèle l'est davantage.

Trois niveaux d'isolation, et ce que chacun laisse passer

Niveau	Ce qu'il coûte	Ce qu'il laisse passer
Sous-processus, avec limites et utilisateur dédié	quelques millisecondes	tout le système de fichiers lisible par cet utilisateur, le réseau, les autres processus
Conteneur (namespaces, cgroups, seccomp)	quelques dizaines de millisecondes	le noyau, partagé avec l'hôte ; une faille du noyau traverse la frontière
MicroVM (virtualisation matérielle)	environ 125 ms au démarrage, quelques ms en restaurant un instantané	ce qui passe par les canaux que l'hôte a ouverts, et rien d'autre

Le conteneur est la réponse habituelle, et elle est raisonnable pour du code que l'on a écrit soi-même. Elle repose sur un noyau partagé. Les échappements de conteneur existent, ils sont documentés, et ils sont exactement le genre de chose qu'un modèle peut reproduire depuis sa mémoire d'entraînement sans même chercher à nuire. Pour du code que je n'ai pas relu, je veux une frontière qui ne dépende pas d'une configuration seccomp bien écrite.

Les clés d'API n'ont rien à faire du même côté que le code

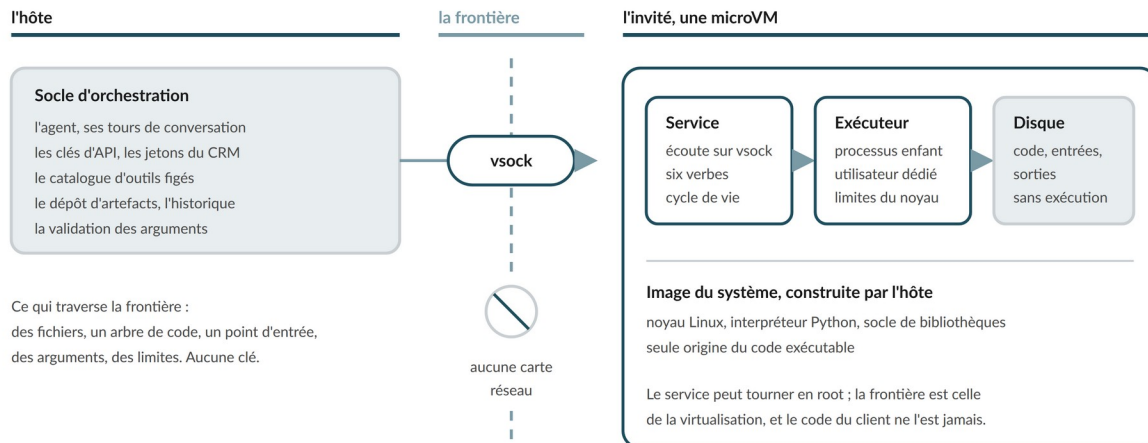
Il y a une deuxième raison, moins visible. Le socle d'orchestration qui fait tourner l'agent porte les clés d'API des modèles, les identifiants du CRM, les jetons d'accès à la boîte mail. Un sous-processus lancé depuis ce socle hérite de son environnement, de ses fichiers, de ses sockets ouverts. Il suffit d'un `os.environ` imprimé dans un fichier de sortie pour que la clé se retrouve dans un document rendu au client. Le code exécuté doit tourner là où ces clés n'existent pas. Une clé absente n'a pas besoin d'être protégée.

3. La microVM comme frontière

Firecracker est un moniteur de machines virtuelles écrit par Amazon pour faire tourner Lambda et Fargate. Il démarre un noyau Linux minimal dans une machine virtuelle dont la surface d'attaque tient en quelques périphériques virtio : un disque, une console, un canal de communication avec l'hôte. Les chiffres publiés par le projet donnent l'échelle : un démarrage en 125 millisecondes, moins de 5 Mio de mémoire par machine au-delà de ce qu'on lui alloue, et une image qui se restaure depuis un instantané en quelques millisecondes.

La vitesse compte moins, pour mon usage, que ce que la frontière permet d'assumer.

- **Aucune interface réseau.** La machine n'a pas de carte réseau, virtuelle ou non. Le code qu'elle exécute ne peut pas appeler une API, exfiltrer un fichier, ni télécharger quoi que ce soit. Il n'y a rien à filtrer, parce qu'il n'y a rien.
- **Un disque de travail séparé du système.** Le système de la machine vient d'une image construite par l'hôte. Les données et le code poussés pour un travail vivent sur un second disque, monté avec les options qui interdisent d'y exécuter quoi que ce soit.
- **Le service invité peut tourner en root.** La frontière de sécurité est celle de la virtualisation. Ce qui se passe à l'intérieur de la machine n'a pas besoin d'être durci avec la même énergie, ce qui simplifie l'écriture du service qui y tourne. Le code du client, lui, ne tourne pas en root, pour une raison différente, que je détaille en partie 7.
- **Une machine par travail, ou une machine réutilisée.** Firecracker permet les deux. Restaurer un instantané pour chaque exécution garantit un état propre à chaque fois. Réutiliser une machine coûte moins cher. Le protocole que je décris ne doit dépendre d'aucun des deux choix.



Ce que la frontière coûte

Une microVM a besoin de KVM, donc d'une machine physique ou d'une instance qui expose la virtualisation imbriquée. Elle a besoin d'une image de système construite à l'avance, avec un noyau, un interpréteur Python, et les bibliothèques que l'on veut rendre disponibles. Elle a besoin d'un orchestrateur côté hôte qui la démarre, lui parle, la tue. Et elle coûte de la mémoire par machine : 256 Mio pour une machine qui fait tourner Python, 512 si on veut que pandas s'y sente à l'aise.

Question ouverte, la première du cahier. À partir de quel volume ce coût est-il justifié ? Pour un agent qui exécute trois traitements par jour pour un carreleur, une machine par travail restaurée depuis un instantané est indolore. Pour un service qui en exécute mille par heure, il faut un bassin de machines chaudes, et le bassin ramène des questions d'état résiduel que la machine jetable avait fait disparaître. Je n'ai pas de réponse chiffrée. J'ai un principe : le protocole entre l'hôte et l'invité doit rester le même dans les deux cas, pour que la décision se prenne plus tard, sur des mesures.

4. Un protocole minimal entre l'hôte et l'invité

L'hôte et l'invité communiquent par vsock, un socket qui traverse la frontière de virtualisation sans passer par le réseau. C'est le seul canal ouvert. Ce qui transite dessus définit exactement ce que l'agent peut faire faire à la machine, et la liste mérite d'être courte.

Ce dont l'hôte a besoin tient en quatre gestes : déposer des fichiers d'entrée ; déposer un arbre de code ; demander l'exécution d'une fonction de cet arbre, avec des arguments et des limites ; récupérer la valeur de retour et les fichiers produits. J'y ajoute un salut au début, pour connaître la version du protocole et les plafonds de la machine, et une remise à zéro, pour le cas où la machine serait réutilisée.

Geste	Ce qu'il transporte	Sens des octets
saluer	version du protocole, plafonds de ressources	contrôle seul
déposer un fichier	un morceau d'un fichier d'entrée	vers l'invité
déposer du code	un morceau d'un fichier source	vers l'invité
exécuter	point d'entrée, arguments nommés, limites	contrôle seul
lire une sortie	un morceau d'un fichier produit	vers l'hôte
remettre à zéro	rien	contrôle seul

Six verbes, dont deux seulement transportent des octets vers l'invité et un seul vers l'hôte. Cette surface est délibérément close. Chaque verbe ajouté plus tard devrait se justifier par une capacité que ces six ne couvrent pas, et non par une commodité côté client. Une méthode « lister les fichiers produits » serait commode. Elle est inutile, parce que l'exécution retourne déjà le manifeste de ce qu'elle a produit, avec la taille et l'empreinte de chaque fichier, et l'hôte décide ensuite ce qu'il rapatrie. Un fichier de 2 Gio qu'il ne veut pas ne coûte aucun transfert.

Pourquoi pas HTTP

La réponse habituelle serait d'embarquer un petit serveur HTTP dans la machine. C'est ce que je voudrais éviter. Un serveur HTTP dans l'invité signifie une pile réseau dans l'invité, donc une interface, donc une surface. Il signifie aussi de l'encodage : les fichiers binaires passeraient en base64 ou en multipart, et chaque octet du fichier traverserait un analyseur de texte. Un cadrage binaire simple sur vsock fait le même travail. Un entier pour la longueur d'un en-tête JSON, l'en-tête, un entier pour la longueur du corps, le corps. Aucun encodage, aucune bibliothèque, une centaine de lignes de chaque côté. Un en-tête de plus de 1 Mio ou un corps de plus de 8 Mio est un défaut de protocole, et la connexion se ferme sans négocier.

Séquentiel, et sans diffusion de la sortie

Le protocole que je propose est strictement séquentiel. Une requête, une réponse, dans l'ordre. Pas de multiplexage, pas de messages non sollicités venant de l'invité. La conséquence est qu'un travail long est opaque jusqu'à sa fin : l'hôte ne voit pas la sortie standard défiler, il la reçoit d'un bloc avec le résultat.

C'est un choix que j'assume, et c'est le deuxième que j'inscris dans la liste des questions ouvertes. Un traitement qui tourne trente secondes sur un fichier de dix mille lignes n'a pas besoin d'être suivi en direct. Un traitement qui tourne cinq minutes, peut-être. La diffusion de la sortie casserait la simplicité du séquençement, et je préfère la laisser hors

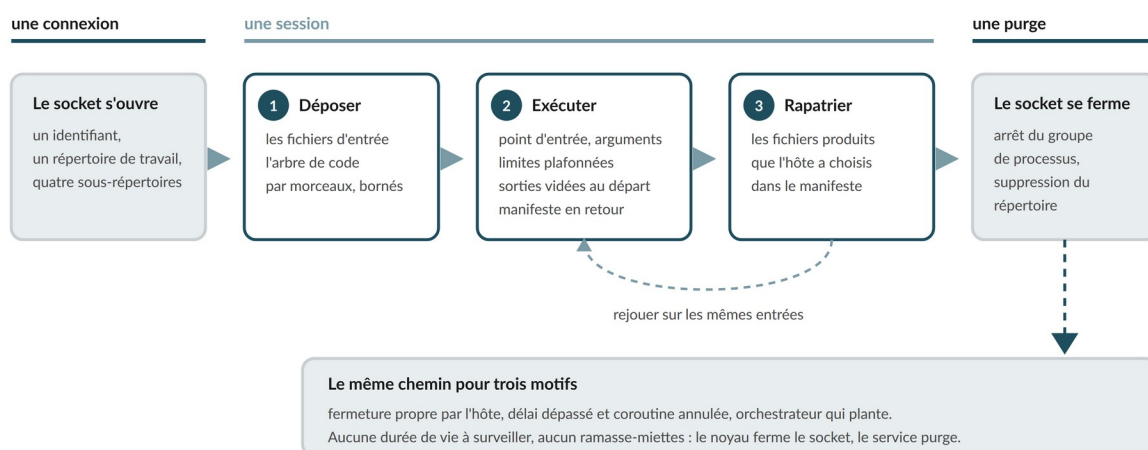
du premier jet plutôt que de concevoir le protocole autour d'un besoin que je n'ai pas encore rencontré.

Il reste un cas que le protocole séquentiel ne bloque pas. Un outil qui, pendant son exécution, aurait besoin d'une capacité de l'hôte, une recherche documentaire par exemple, peut ouvrir de lui-même une connexion vers l'hôte sur un autre canal vsock, indépendant de la session. L'invité reste sans réseau, et l'hôte décide ce qu'il expose sur ce canal. C'est un point que je note sans le développer ; il mériterait un cahier à lui seul.

5. Lier la session à la connexion

C'est l'idée centrale de ce cahier, et la plus simple. Une connexion vsock ouverte est une session. Une connexion fermée est une purge.

À l'acceptation de la connexion, l'invité crée un répertoire de travail, avec quatre sous-répertoires : le code, les entrées, les sorties, et un répertoire courant pour ce que le traitement voudra écrire de temporaire. À la fermeture du socket, que ce soit une fermeture propre, une réinitialisation, ou la disparition pure et simple de l'hôte, l'invité tue le groupe de processus du travail en cours et supprime le répertoire.



Il n'y a pas de méthode « ouvrir une session » ni « fermer une session ». Il n'y a pas de durée de vie à surveiller, pas de ramasse-miettes à écrire, pas de session zombie. Le nettoyage est porté par le noyau. Si l'orchestrateur côté hôte plante en plein traitement, le socket se ferme, et la purge se déclenche sans qu'aucune ligne de code de reprise n'ait été écrite.

Ce que ça règle du côté de l'hôte

Le socle d'orchestration applique un délai maximal à chaque appel d'outil, trente secondes par défaut dans le mien. Quand ce délai expire, la coroutine est annulée. Si la fermeture du socket est dans un bloc finally, l'annulation ferme le socket, la fermeture purge la machine, et l'exécution qui tournait encore est tuée. Le cas du délai dépassé est

couvert par la même mécanique que le cas nominal. Il n'y a pas de chemin d'abandon distinct à tester.

Il faut en échange que le délai côté hôte couvre tout : le temps d'exécution accordé au traitement, plus l'allocation de la machine, plus le transfert du code et des entrées, plus le rapatriement des sorties. Un délai de trente secondes pour un traitement à qui l'on a accordé trente secondes est une erreur de composition. L'hôte abandonnera pendant que la machine travaille encore.

Les exécutions répétées

Une session peut demander plusieurs exécutions, avec des arguments différents, sur les mêmes entrées. C'est utile pour rejouer un traitement après une correction sans tout retransférer. Le répertoire de sortie est vidé au début de chaque exécution, pour que le manifeste rendu décrive exactement ce que cette exécution a produit, et jamais un cumul. Un client qui veut garder les sorties d'une exécution les rapatrie avant de relancer.

Question ouverte, la troisième. Y a-t-il des cas d'usage qui auraient besoin d'une session qui survive à la connexion ? Un traitement en plusieurs étapes, lancé par des tours de conversation différents, avec un état intermédiaire volumineux ? Je n'en ai pas rencontré dans mes agents. Si le cas se présente, la réponse la plus propre me semble être de rapatrier l'état sur l'hôte entre les deux tours, plutôt que d'introduire une notion de session persistante qui ramènerait avec elle tout ce que la liaison à la connexion avait fait disparaître.

6. Le problème des dépendances

Un traitement écrit par le modèle importera des bibliothèques. csv et json viennent avec Python. pandas ne vient pas avec Python, et c'est pourtant ce que le modèle écrira dès qu'il verra un fichier Excel. La question de savoir d'où viennent les bibliothèques est celle qui décide de la moitié de la sécurité de l'ensemble.

Il y a trois origines possibles pour du code qui s'exécute dans la machine. La bibliothèque standard, qui vient avec l'interpréteur de l'image. Un socle de bibliothèques installé dans l'image au moment de sa construction, sur l'hôte, à partir de paquets résolus par un gestionnaire de paquets. Et le code poussé à la volée par le canal de transfert, celui que le modèle vient d'écrire.

Le critère est la provenance

Ces trois origines se distinguent par qui les a produites et par quel chemin elles sont arrivées, davantage que par ce qu'elles contiennent. L'image est construite par l'hôte, avec des paquets dont l'hôte a vérifié la provenance ; on lui fait confiance au même titre qu'au système lui-même. Le canal de transfert reçoit ce que le modèle a écrit ; on ne lui fait aucune confiance, et c'est le point de départ de tout le design.

Origine des octets	Qui les a produits	Confiance	Montage
Image du système	l'hôte, à la construction	établie	exécutable
Socle de bibliothèques	l'hôte, par le gestionnaire de paquets	établie	exécutable, en lecture seule
Disque de travail	le modèle, ou le traitement lui-même	nulle	sans exécution

Le disque de travail est monté avec l'option qui interdit d'y exécuter du code binaire. Python lit des fichiers .py sans bit d'exécution, donc un traitement écrit en Python fonctionne normalement. En revanche, une extension compilée déposée là ne peut pas être chargée : l'appel système qui la projette en mémoire échoue. Le code natif ne peut venir que de l'image, résolu sur l'hôte, c'est-à-dire du même canal de confiance que le système.

Ce refus mérite d'être fait au bon endroit. Si l'on laisse le montage produire l'échec, le modèle reçoit un message de projection mémoire qui n'oriente vers rien. Si le canal de transfert refuse d'écrire un fichier .so dès son dépôt, avec un message qui dit que le code natif vient de l'image et non du transfert, le modèle comprend ce qu'il doit changer. Le chemin était valide, l'intention légitime ; c'est une contrainte de plateforme, et le message doit le dire.

Le cas piège

« Pur Python » ne se lit pas au nom du paquet. pydantic tire pydantic-core, qui est du Rust compilé. orjson, cryptography, lxml, pillow sont natifs. numpy et pandas le sont évidemment. Un modèle qui écrit `import pandas` ne demande pas un fichier Python ; il demande une quinzaine de Mio de code compilé. Si pandas n'est pas dans l'image, le traitement échouera à l'import, et le message d'erreur doit le dire au bon endroit : sur l'hôte, à l'enregistrement du traitement, en analysant ses imports, plutôt qu'au premier appel dans la machine.

Ce qui reste au canal de transfert, c'est le code du traitement lui-même et, si besoin, un petit paquet Python pur transporté avec lui. Un module utilitaire de quelques dizaines de kilooctets se transporte sans problème. Un arbre de dépendances complet, httpx et ses transitives par exemple, pèse 5 Mio et fait du transfert le poste dominant d'un traitement qui s'exécute en 200 millisecondes. Un plafond sur la taille du code transféré, 16 Mio dans mon hypothèse, borne cette dérive sans gêner l'usage prévu.

Question ouverte, la quatrième. Qui décide du contenu du socle, et comment le fait-on évoluer ? Chaque bibliothèque ajoutée élargit ce que les traitements peuvent faire, et alourdit l'image, donc l'instantané, donc le démarrage. Changer le socle invalide tous les instantanés existants. Ma liste de départ tiendrait en sept noms : httpx, numpy, pandas,

pillow, lxml, python-dateutil, orjson. Je ne sais pas encore si c'est trop ou trop peu, et je ne le saurai qu'en regardant ce que les modèles importent réellement sur des demandes réelles.

7. Limites de ressources, et le mot « plafond »

Un traitement écrit par un modèle peut boucler. Il peut allouer une liste de cent millions d'éléments parce qu'il a mal lu la taille du fichier. Il peut écrire un fichier de sortie jusqu'à remplir le disque. Aucun de ces cas n'est malveillant, et tous doivent être arrêtés avant de nuire à la machine, et donc aux traitements suivants si la machine est réutilisée.

Limite	Défaut	Plafond	Mécanisme
Temps réel	30 s	5 min	minuteur côté service, puis arrêt forcé du groupe de processus
Temps processeur	30 s	5 min	limite du noyau sur le processus
Mémoire	512 Mio	768 Mio	limite du noyau sur l'espace d'adressage
Taille d'un fichier	256 Mio	1 Gio	limite du noyau
Fichiers produits	128	512	comptage au relevé des sorties
Sortie standard capturée	256 Kio		tampon borné, tête et queue conservées

L'hôte propose, la machine plafonne

Le principe qui organise cette table tient en une phrase. L'hôte envoie des limites avec chaque demande d'exécution ; la machine les compare à ses plafonds et retient la plus basse des deux, sans erreur, en renvoyant la valeur effectivement appliquée. Un traitement ne peut pas s'octroyer trente minutes en le demandant. Le plafond est dans l'image, il a été posé par la personne qui l'a construite, et aucune requête ne le déplace.

Deux limites de temps sont nécessaires. La limite sur le temps processeur ne voit pas un traitement qui dort, ou qui attend une lecture. Le minuteur en temps réel ne distingue pas une boucle serrée d'un traitement légitimement lent. Les deux ensemble couvrent les deux cas.

La mémoire a un plancher. En dessous de 128 Mio environ, l'interpréteur Python échoue à importer sa propre bibliothèque standard, et le message d'erreur qui en résulte est illisible. Une demande en dessous du plancher doit être refusée avec un message clair plutôt qu'honorée.

La sortie standard est tronquée par le milieu. Quand un traitement écrit plus que ce que le tampon accepte, ce sont la tête et la queue qui sont conservées, avec un marqueur entre

les deux. L'information de diagnostic se trouve aux deux extrémités d'une sortie, presque jamais au centre.

Le processus qui exécute n'est pas le service

Le service qui écoute sur vsock ne doit pas exécuter le code lui-même. Il lance un processus enfant, sous un utilisateur dédié sans droits, dans son propre groupe de processus, avec les limites posées avant l'exécution et l'interdiction d'acquérir de nouveaux privilèges. Trois raisons à cela : les limites du noyau se posent sur un processus, un fil d'exécution ne peut pas les porter ; un dépassement de délai doit pouvoir tuer sans coopération ; une extension compilée qui plante ne doit pas emporter le service avec elle.

Le signal d'arrêt vise le groupe de processus entier. Un traitement qui a lui-même lancé un sous-processus laisserait sinon un orphelin, qui continuerait à consommer de la mémoire dans une machine réutilisée. C'est le genre de détail qui ne se voit pas en démonstration et qui se voit très bien le troisième jour.

8. Ce que l'agent doit voir quand ça échoue

Le vrai destinataire du message d'erreur n'est pas un humain. C'est le modèle qui vient d'écrire le traitement et qui va tenter de le corriger au tour suivant. Cette phrase change la façon d'écrire les erreurs, et j'en tire deux exigences.

Isoler le nom du module manquant

Quand un import échoue, l'exception Python porte le nom du module dans un attribut dédié. Ce nom doit remonter dans un champ à part du message d'erreur, plutôt qu'enfoui dans un texte. C'est ce qui permet à l'hôte de décider, sans lire le texte, si ce module doit rejoindre le socle de l'image, être transporté avec le code, ou entraîner le refus du traitement. C'est aussi ce qui permettra, plus tard, de construire une liste des imports autorisés à partir de ce que les traitements demandent réellement.

Nettoyer la pile

Quand le traitement lève une exception, la pile d'appels contient les cadres de l'exécuteur, ceux de runpy, ceux d'importlib, puis ceux du code du modèle. Un modèle qui voit dans sa pile un fichier qui ne lui appartient pas part corriger le mauvais fichier. Il faut retirer les cadres qui ne sont pas les siens, et rendre les chemins relatifs au répertoire de code. Le modèle ne doit voir que ce qu'il a écrit.

```
{
  "ok": false,
  "error": {
    "kind": "tool_raised",
    "message": "colonne 'client' absente du fichier",
    "detail": {
      "type": "KeyError",
      "traceback": [
        "File \\recap/main.py\\", line 12, in run",
        "    total = lignes[\\\"client\\\"]"
      ]
    }
  }
}
```

Le champ `kind` est un identifiant stable et énumérable. Une dizaine de valeurs suffisent : erreur de protocole, chemin refusé, code natif refusé, code invalide, import échoué, exception du traitement, délai dépassé, mémoire dépassée, limite franchie, résultat non sérialisable, défaillance de l'exécuteur. Le champ `message` est écrit pour le modèle. Le champ `detail` est structuré et dépend du `kind`.

Le code de sortie ne dit pas si le traitement a réussi

Un piège que j'ai vu ailleurs : l'exécuteur écrit son résultat dans un fichier et sort avec un code. Il est tentant de lire le code de sortie comme un succès ou un échec du traitement. C'est faux. Un traitement qui lève une exception produit un résultat valide, avec `ok` à faux et l'exception décrite, et l'exécuteur sort avec zéro. Le code de sortie ne dit qu'une chose : si le fichier de résultat est exploitable. Confondre les deux ferait remonter toute erreur métier comme une défaillance de l'exécuteur, et le modèle perdrait le diagnostic dont il a besoin.

Question ouverte, la cinquième. Jusqu'où structurer l'erreur ? Chaque champ ajouté est un champ que l'hôte doit comprendre, et le contrat s'alourdit. Le nom du module isolé et la pile nettoyée me semblent indispensables. Le reste, je ne le sais pas.

9. Les artefacts, et une décision qu'on ne reprend pas

Le récapitulatif du carreleur est un fichier. Un outil dans un socle d'orchestration rend une chaîne de caractères, dont la taille est bornée, et cette chaîne revient dans le contexte du modèle. Un traitement qui produit un tableur de 2 Mio n'a nulle part où le mettre dans sa valeur de retour.

Les octets sortent donc de la machine par le verbe de lecture des sorties, et sont confiés à un dépôt d'artefacts côté hôte. Sur le disque de l'hôte pour commencer, avec un chemin qui reproduit la forme d'une clé d'object store, pour que la bascule vers un stockage distant reste mécanique le jour où elle sera nécessaire.

```
artefacts/<client>/<identifiant d'exécution>/recap-chantiers.xlsx
artefacts/<client>/<identifiant d'exécution>/_manifest.json
```

Le segment du client figure en tête même quand il n'y a qu'un seul client. C'est lui qui permettra une règle d'accès par préfixe sur un stockage distant, et le réintroduire plus tard imposerait de tout recopier. La rétention se fait par une passe au démarrage du dépôt, qui supprime les répertoires d'exécution plus vieux que trente jours. C'est ce qu'une règle de cycle de vie ferait toute seule sur un stockage distant, et ne rien mettre à la place au moment de la bascule est indolore.

Ce que le modèle voit

Le traitement a produit un fichier. Le modèle doit pouvoir en parler au client, le proposer, le réutiliser dans une étape suivante. Ce qu'il reçoit est une référence opaque.

```
artifact://client-0412/exec-8f3a/recap-chantiers.xlsx
```

Jamais un chemin de système de fichiers, et la raison est celle de l'historique. Le socle d'orchestration persiste la conversation et la relit au tour suivant, puis la résume quand elle devient trop longue. Un chemin absolu qui entre dans cet historique y reste gravé, survit au résumé, survit au déménagement du projet, survit au changement de stockage. C'est la seule décision de ce cahier qui coûte cher à reprendre, et c'est pour cela que je la fixe avant d'écrire quoi que ce soit d'autre.

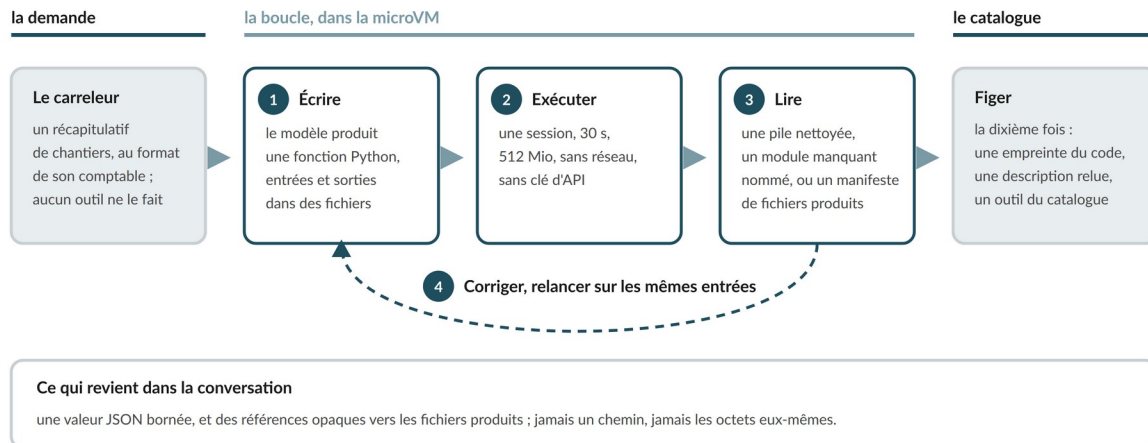
Deux règles reprises d'ailleurs

- **L'écriture est atomique.** Un fichier temporaire à nom unique, une synchronisation sur disque, un renommage. Un artefact à moitié écrit doit rester invisible ; un artefact visible et tronqué est pire qu'un artefact absent.
- **Les noms sont encodés.** Les noms d'artefacts viennent du répertoire de sortie, donc de code écrit par le modèle. La validation des chemins dans la machine protège la machine. L'hôte doit se protéger séparément, avec sa propre validation, sans supposer que l'invité l'a fait.

10. Ce que ça change pour l'agent

Retour au carreur. Avec ce design, la demande du récapitulatif suit un chemin qui n'existait pas.

Le modèle écrit le traitement. Le socle d'orchestration ouvre une session, pousse le fichier d'export du logiciel de devis, pousse le code, demande l'exécution avec trente secondes et 512 Mio. Le traitement lève une exception à la ligne 12, parce que le modèle a supposé une colonne qui n'existe pas. Le modèle lit la pile, nettoyée, corrige, relance sur les mêmes entrées. La deuxième exécution produit un fichier. Le socle le rapatrie, le dépose dans les artefacts, et rend au modèle une référence. Le modèle répond au carreur avec un lien.



Pour l'orchestrateur, rien de tout cela n'est visible. Un traitement exécuté en microVM est un outil comme un autre, une nature de plus à côté de la fonction locale, du serveur MCP, du modèle spécialisé et de la recherche documentaire que je décrivais dans le cahier 03. Il a un nom, une description, un schéma d'entrée, et une méthode qui s'exécute. Ce que la méthode fait, ouvrir une session et dérouler six verbes, ne regarde qu'elle. Les arguments sont validés contre le schéma avant qu'une machine soit allouée, comme pour n'importe quel outil ; la validation dans l'invité reste utile en défense en profondeur, mais elle n'est pas la ligne principale.

Du traitement jetable à l'outil figé

Ce qui m'intéresse le plus dans cette boucle n'est pas la première exécution. C'est la dixième. Si le récapitulatif pour le comptable est demandé chaque mois, le traitement que le modèle a écrit la première fois et corrigé une fois est, de fait, un outil. Il a un code stable, des entrées connues, une sortie attendue. Il pourrait rejoindre le catalogue, avec un nom, une description écrite par le modèle et relue par un humain, et une empreinte de son code qui garantit que ce qui s'exécute le mois prochain est ce qui s'est exécuté ce mois-ci.

Le contenu du répertoire de code déterminerait cette empreinte. Un paquet transporté avec le code en fait donc partie, et deux exécutions de la même empreinte ont strictement le même arbre de dépendances. C'est un argument de plus pour transporter les petites dépendances avec le code plutôt que par un canal séparé.

La validation à l'enregistrement, l'analyse des imports, la liste des imports autorisés, la collision entre un module transporté et un module du socle : tout cela relève de cette étape d'enregistrement, côté hôte, et non du service qui exécute. Le service applique une décision prise ailleurs. Cette séparation me semble être la plus importante du design, et c'est celle que je surveillerais le plus au moment de l'écrire, parce que c'est celle que la commodité érode en premier.

11. Ce que cette hypothèse ne couvre pas

Un cahier de réflexion se termine sur ce qu'il n'a pas résolu. Je rassemble ici les questions ouvertes semées dans les parties précédentes, et j'y ajoute celles qui n'avaient leur place nulle part.

Les questions posées en chemin

- **Une machine par travail, ou un bassin.** Le protocole est neutre. Le coût ne l'est pas. La réponse viendra de mesures que je n'ai pas.
- **La diffusion de la sortie standard.** Exclue du premier jet pour garder le séquençement trivial. Compatible avec le cadrage si le besoin apparaît.
- **Une session qui survit à la connexion.** Aucun cas rencontré. Si l'un se présente, rapatrier l'état sur l'hôte plutôt qu'introduire une persistance dans l'invité.
- **Le contenu du socle de bibliothèques.** Sept noms pour commencer, et une observation de ce que les modèles importent réellement.
- **Jusqu'où structurer l'erreur.** Deux champs indispensables. Le reste à l'usage.

Ce que le cahier n'a pas abordé

La sérialisation du résultat. Un traitement rend une valeur, et cette valeur doit devenir du JSON. Faut-il un encodeur strict, qui refuse un datetime ou un Decimal, ou un encodeur tolérant qui les convertit ? Le tolérant simplifie l'écriture des traitements. Le strict rend le contrat plus net pour le modèle. Je penche pour le strict, sans certitude.

La vérification avant exécution. Un linter, un vérificateur de types, une suite de tests écrite par le modèle lui-même, exécutés dans la machine avant le vrai lancement. Même mécanique que l'exécution, mais elle exige ces outils dans l'image, et elle relève du pipeline d'enregistrement.

La granularité du client. Une machine par client, une image par client, ou une machine partagée avec un répertoire par session ? Le cahier suppose que chaque session est isolée, et que deux clients ne partagent jamais une session. Il ne dit pas si deux clients peuvent partager une machine, et la réponse dépend de ce que l'on est prêt à promettre contractuellement.

L'injection indirecte. Un fichier d'entrée peut contenir des instructions destinées au modèle. Le traitement écrit par le modèle peut avoir été influencé par ces instructions. La microVM borne ce que ce traitement peut faire ; elle ne borne pas ce qu'il produit, et un fichier de sortie qui contient un texte trompeur reviendra vers le client. C'est le même angle mort que dans le cahier 04, et il n'est pas traité ici non plus.

L'auditabilité. C'est la question de fond. Un catalogue d'outils figés se relit avec le client, se restreint, se signe. Un agent qui écrit ses propres outils produit un code différent à

chaque demande. Ce code s'exécute dans une frontière que je crois solide. Mais qui le relit ? Qui répond quand le récapitulatif du comptable contient une erreur de calcul que ni le modèle ni le carreur n'ont vue ? La microVM répond à la question « que peut faire ce code ». Elle ne répond pas à la question « ce code fait-il ce qu'il faut ». Cette seconde question est celle que le catalogue figé résolvait par la relecture humaine, et je n'ai pas encore de réponse équivalente pour un outil écrit à la volée. Le passage du traitement jetable à l'outil figé, avec une relecture au moment de figer, en est peut-être une. Je ne l'ai pas éprouvée.

12. Ce que je retiens

- La frontière entre ce qu'un agent peut écrire lui-même et ce qu'un humain doit livrer est celle de l'effet de bord. Lire des entrées et produire des sorties peut être écrit par le modèle. Toucher au monde reste dans le catalogue.
- Le code produit par un modèle est du code non fiable au sens strict, même sans intention malveillante. Il s'exécute là où les clés d'API n'existent pas, derrière une frontière qui ne dépend pas d'une configuration bien écrite.
- Une microVM sans interface réseau permet d'assumer des choses qu'un conteneur oblige à vérifier. Le service qui y tourne peut rester simple.
- Six verbes suffisent au protocole. Chaque verbe ajouté doit se justifier par une capacité que les six ne couvrent pas.
- Lier la session à la connexion fait porter le nettoyage par le noyau. Le crash de l'hôte, le délai dépassé et la fermeture propre passent par le même chemin.
- Le critère de confiance est la provenance des octets. Ce qui vient de l'image s'exécute. Ce qui vient du canal de transfert ne s'exécute qu'en tant que Python interprété, sur un disque sans exécution.
- L'hôte propose des limites, la machine plafonne. Deux limites de temps, un plancher de mémoire, un arrêt qui vise le groupe de processus.
- Le destinataire du message d'erreur est le modèle. Le nom du module manquant est isolé, la pile est nettoyée de tout ce qui n'est pas le code du modèle.
- Le modèle reçoit des références opaques vers les fichiers produits, jamais des chemins. C'est la seule décision de ce cahier qui coûte cher à reprendre.
- La microVM répond à « que peut faire ce code ». Elle ne répond pas à « ce code fait-il ce qu'il faut ». Cette question reste ouverte.

13. À propos

Je conçois et j'intègre des agents IA destinés à tourner en production, pour des PME et des artisans. Ce cahier est le premier de la collection à décrire un design plutôt qu'un outil en

service. Il vient d'une demande réelle, formulée au téléphone par un client réel, à laquelle je n'ai pas encore de réponse déployée.

J'écris et je maintiens un socle d'orchestration public. Il porte les mécanismes décrits dans cette collection : validation des arguments avant appel, exécution parallèle bornée, checkpoints et reprise. Le service décrit ici s'y brancherait comme une nature d'outil supplémentaire.

Le code est public : github.com/denislamard/loom

Le détail technique et les cas d'usage : denislamard.github.io